Problem definition

This project implements and compares mainly 2 strategies: KIRIS and RAG_GPT on the recognising and coding of medical entities in clinical stroke texts, using SNOMED-CT terminology.

In my machine, I get the following results

```
Strategy                      F1-Score   Precision  Recall     Pred   Match
Time

-------------------------------------------------------------------------------
---------------------------------------

01_KIRIs                      0.8000     0.8381     0.7652     105    88
0.035     s
04_RAG_GPT                    0.5983     0.6604     0.5469
```

My benchmark are clinical sentences, which should give me 32 concepts. I am testing the RAG_GPT with a ontology of those 32 concepts + 2470 extra medical concepts to add sound to the model. I should be able to get at least 0.65-0.7 with the RAG_GPT approach.

This is the output of my last execution:
[OK] Procesamiento completado: 27 entidades
[Pipeline] [OK] Completado: 72 predicciones generadas

=== RAG+GPT Predictions ===
note_id start end concept_id span_text confidence anatomy_code presence_code
1 75 87 38341003 hypertension 0.85 12738006 52101004
1 92 109 73211009 diabetes mellitus 0.85 12738006 52101004
1 144 163 13791008 left-sided weakness 0.85 12738006 52101004
1 287 349 450893003 National Institutes of Health Stroke Scale (NIHSS) score of 12 0.85 12738006 52101004
1 356 386 50582007 left hemiparesis (4/5 strength 0.85 12738006 52101004
1 412 427 103676007 mild dysarthria 0.85 12738006 52101004
1 548 610 26036001 occlusion of the right middle cerebral artery (MCA) M1 segment 0.85 69930009 52101004
1 612 665 450893003 Alberta Stroke Program Early CT Score (ASPECTS) was 8 0.85 12738006 52101004
1 689 703 230691006 large penumbra 0.85 12738006 52101004
1 709 744 103668005 core-penumbra mismatch ratio of 2.4 0.85 12738006 52101004
1 784 829 387467008 intravenous tissue plasminogen activator (tPA 0.85 12738006 52101004
1 874 896 26036001 large vessel occlusion 0.85 12738006 52101004
1 695 703 230691006 penumbra 0.85 12738006 52101004
1 923 946 433112001 mechanical thrombectomy 0.85 12738006 52101004
1 1006 1056 450893003 Thrombolysis in Cerebral Infarction (TICI) 2B flow 0.85 12738006 52101004
1 962 987 449894001 Successful recanalization 0.85 12738006 52101004

1 1131 1165 69449002 minimal hemorrhagic transformation 0.85 12738006 52101004
1 1235 1256 450893003 NIHSS decreasing to 4 0.85 12738006 52101004
2 40 59 49436004 atrial fibrillation 0.85 732006008 52101004
2 144 159 25064002 severe headache 0.85 182326009 52101004
2 161 167 422587007 nausea 0.85 12738006 52101004
2 173 181 422400008 vomiting 0.85 12738006 52101004
2 151 159 25064002 headache 0.85 182326009 52101004
2 443 458 182368009 nuchal rigidity 0.85 182326009 52101004
2 476 501 450893003 Glasgow Coma Scale was 14 0.85 12738006 52101004
2 552 575 50960005 subarachnoid hemorrhage 0.85 263353005 52101004
2 581 629 464005 blood in the basal cisterns and sylvian fissures 0.85 369389009 52101004
2 661 723 432101006 7mm anterior communicating artery aneurysm with a daughter sac 0.85 69930009 52101004
2 949 985 433112001 Endovascular coiling of the aneurysm 0.85 432101006 52101004
2 1072 1097 230690007 delayed cerebral ischemia 0.85 263353005 52101004
2 1121 1141 38341003 induced hypertension 0.85 12738006 52101004
2 1146 1165 26036001 balloon angioplasty 0.85 12738006 52101004
2 1196 1228 450893003 modified Rankin Scale score of 2 0.85 12738006 52101004
3 64 78 4647008 hyperlipidemia 0.85 12738006 52101004
3 115 135 13791008 right-sided weakness 0.85 12738006 52101004
3 336 355 50582007 unilateral weakness 0.85 12738006 52101004
3 567 613 55342001 small acute infarct in the left corona radiata 0.85 727323009 52101004
3 799 829 230690007 Transient ischemic attack (TIA 0.85 12738006 52101004
3 836 866 55342001 completed small vessel infarct 0.85 279605007 52101004
3 886 911 86501000 artery-to-artery embolism 0.85 12738006 52101004
3 1049 1071 86547008 carotid endarterectomy 0.85 86547008 52101004
4 0 19 433112001 THROMBECTOMY REPORT 0.85 12738006 52101004
4 87 94 87486003 aphasia 0.85 12738006 52101004
4 99 115 50582007 right hemiplegia 0.85 12738006 52101004
4 191 209 450893003 NIHSS score was 18 0.85 12738006 52101004
4 251 269 370640001 ASPECTS score of 9 0.85 182326009 52101004
4 295 335 26036001 left M1 middle cerebral artery occlusion 0.85 69930009 52101004
4 341 368 449894001 good collateral circulation 0.85 12738006 52101004
4 370 388 450893003 collateral score 3 0.85 12738006 52101004
4 455 475 314355009 right femoral artery 0.85 12738006 52101004
4 519 547 86547008 left internal carotid artery 0.85 12738006 52101004
4 557 582 449894001 stent retriever technique 0.85 12738006 52101004
4 588 592 88611005 clot 0.85 12738006 52101004
4 274 285 77343006 angiography 0.85 12738006 52101004
4 656 677 266257000 TICI 3 recanalization 0.85 12738006 52101004
4 731 755 450893003 neurointensive care unit 0.85 12738006 52101004
4 663 677 449894001 recanalization 0.85 12738006 52101004
4 836 855 450893003 NIHSS improved to 8 0.85 12738006 52101004
5 0 28 230690007 POSTERIOR CIRCULATION STROKE 0.85 361284004 52101004
5 54 83 26036001 vertebrobasilar insufficiency 0.85 26036001 52101004

5 114 121 245647007 vertigo 0.85 12738006 52101004
5 137 143 20262006 ataxia 0.85 12738006 52101004
5 257 295 57516003 bilateral internuclear ophthalmoplegia 0.85 730424005 52101004
5 301 315 20262006 truncal ataxia 0.85 361295005 52101004
5 317 333 20262006 Cerebellar signs 0.85 314166009 52101004
5 354 363 20262006 dysmetria 0.85 12738006 52101004
5 368 386 20262006 dysdiadochokinesia 0.85 12738006 52101004
5 450 464 55342001 acute infarcts 0.85 28390009 52101004
5 612 650 727323009 poor posterior circulation collaterals 0.85 361284004 52101004
5 730 751 67889009 pattern of infarction 0.85 12738006 52101004
5 772 803 26036001 basilar artery branch occlusion 0.85 360537008 52101004
5 1086 1106 55342001 small infarct volume 0.85 12738006 52101004

[run_rag_gpt] Total predictions: 72

==========
==========
==========

RESULTS - RAG+GPT

[METRICS] METRICS:
Precision: 0.6604
Recall: 0.5469
F1-Score: 0.5983
Coverage: 1.0000

[CHART] COUNTS:
Predictions: 53
Exact Matches: 35
Partial Matches: 18
Ground Truth: 64

[TIME] EXECUTION TIME: 159.70 seconds

This is all my code:

.env:

RAG Configuration

RAG_TOP_K=30
RAG_THRESHOLD=0.50
RAG_MAX_DISPLAY=12

RAG_QUERY_SUFFIX=disorder finding
RAG_ALLOW_FALLBACK=false

LLM Configuration

RAG_USE_LLM_VALIDATION=false
RAG_LLM_MODEL=gpt-4o
RAG_LLM_TEMPERATURE=0.0

Span Processing

RAG_SPAN_TIGHTEN=true

Evaluation Offsets (for benchmark compatibility)

EVAL_OFFSET_BASE=0
EVAL_END_INCLUSIVE=false

--- Content of ./scripts/build_index.py ---
#!/usr/bin/env python3
"""
Thin wrapper around src/indexer/build_index.py
"""

from **future** import annotations

import sys
from pathlib import Path

ROOT = Path(**file**).resolve().parent.parent
SRC_DIR = ROOT / "src"
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from indexer.build_index import main

if **name** == "**main**":
main()

--- Content of ./scripts/run_rag_gpt.py ---
#!/usr/bin/env python3
"""
CLI: end-to-end pipeline (NER -> RAG -> Coding)

Defaults:

- Input auto-detected from ../../data (repo layout: .../benchmark/strategies/rag_gpt)

- Prints results to terminal by default
- If --results is provided, also writes a CSV there
- At the end, computes F1 metrics using benchmark/evaluation/metric_calculator.py
  """

from **future** import annotations

import argparse
import os
import sys
import time
from pathlib import Path
import pandas as pd

Path setup

ROOT = Path(**file**).resolve().parent.parent # .../strategies/rag_gpt
SRC_DIR = ROOT / "src"
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

evaluation (.../benchmark/evaluation/metric_calculator.py)

EVAL_DIR = ROOT.parent.parent / "evaluation"
if EVAL_DIR.exists() and str(EVAL_DIR) not in sys.path:
sys.path.insert(0, str(EVAL_DIR))

Lazy import (will fail gracefully if the file isn't there)

try:
# The project's module is named `metrics_calculator.py` (plural).
# Earlier code attempted to import `metric_calculator` (singular),
# which causes an ImportError even when the file exists under
# benchmark/evaluation. Import the correct module name here.

from metrics_calculator import MetricsCalculator # type: ignore
except Exception:
MetricsCalculator = None # type: ignore

from pipeline import RAGGPTPipeline # noqa: E402

Helpers

---

def _resolve_default_input() -> Path:
"""
Find a sensible default CSV under ../../data (i.e., .../benchmark/data).
Order of preference:
1) notes.csv
2) notes_dev.csv
3) first *.csv in the folder
You can override the folder with BENCHMARK_DATA_DIR.
"""
default_dir = Path(os.getenv("BENCHMARK_DATA_DIR", str(ROOT.parent.parent / "data")))
if not default_dir.exists():
raise FileNotFoundError(f"Default data directory not found: {default_dir}")

```
preferred = ["notes.csv", "notes_dev.csv"]
for name in preferred:
    p = default_dir / name
    if p.exists():
        return p

csvs = sorted(default_dir.glob("*.csv"))
if not csvs:
    raise FileNotFoundError(f"No CSV files found in {default_dir}")
return csvs[0]
```

def _resolve_default_truth() -> Path | None:
"""
Try common ground-truth filenames in ../../data.
"""

data_dir = Path(os.getenv("BENCHMARK_DATA_DIR", str(ROOT.parent.parent / "data")))
if not data_dir.exists():
return None
# First, try a set of exact common filenames
for name in ["ground_truth.csv", "gold.csv", "annotations.csv", "labels.csv"]:
p = data_dir / name
if p.exists():
return p

```
# If none of the exact names match, try more flexible substring matches
# to accept names like 'train_annotations.csv' or 'mimic_gold.csv'.
keywords = ["ground", "gold", "annotation", "annotations", "label"]
for f in sorted(data_dir.glob("*.csv")):
    fname = f.name.lower()
    if any(k in fname for k in keywords):
        return f
return None
```

def _read_input_csv(path: Path) -> pd.DataFrame:
df = pd.read_csv(path)
if "note_id" not in df.columns and "id" in df.columns:
df = df.rename(columns={"id": "note_id"})
if not {"note_id", "text"}.issubset(df.columns):
raise ValueError(
f"Input CSV must contain columns: note_id,text (got: {list(df.columns)})"
)
return df

def _read_truth_csv(path: Path) -> pd.DataFrame:
"""
Read ground-truth and standardize to have at least: note_id, concept_id
Accept a few common aliases for concept_id.
"""
df = pd.read_csv(path)
if "note_id" not in df.columns:
raise ValueError(f"Ground truth must include 'note_id' (got: {list(df.columns)})")

```
# Map possible aliases to 'concept_id'
alias_map = {
    "concept": "concept_id",
    "code": "concept_id",
    "snomed": "concept_id",
    "snomed_code": "concept_id",
    "gold_concept_id": "concept_id",
    "label": "concept_id",
```

```
        "target": "concept_id",
    }
    cols_lower = {c.lower(): c for c in df.columns}
    if "concept_id" not in df.columns:
        for alias_lower, target in alias_map.items():
            if alias_lower in cols_lower:
                df = df.rename(columns={cols_lower[alias_lower]: target})
                break

    if "concept_id" not in df.columns:
        raise ValueError(
            f"Ground truth must include 'concept_id' (got: {list(df.columns)}). "
            f"Consider renaming your SNOMED column to 'concept_id'."
        )

    # Coerce types that the provided MetricsCalculator expects
    df["note_id"] = df["note_id"].astype(int, errors="ignore")
    # leave concept_id as-is; the calculator stringifies when comparing
    return df
```

```
def _print_predictions(preds: pd.DataFrame, max_rows: int = 50) -> None:
cols = [
"note_id", "start", "end", "concept_id",
"span_text", "confidence", "anatomy_code", "presence_code"
]
cols = [c for c in cols if c in preds.columns]
with pd.option_context(
"display.max_rows", max_rows,
"display.max_colwidth", 120,
"display.width", 0
):
print()
print("=== RAG+GPT Predictions ===")
print(preds[cols].to_string(index=False))
print()
print(f"[run_rag_gpt] Total predictions: {len(preds)}")
```

Main

---

def main():
# Accept a compatibility alias: `-results` -> `--results`
argv = ["--results" if a == "-results" else a for a in sys.argv[1:]]

```
ap = argparse.ArgumentParser()
ap.add_argument("--input", help="Input CSV with columns: note_id,text (defaults
to ../../data/*)")
ap.add_argument("--results", help="If provided, write predictions CSV to this
path")
ap.add_argument("--truth", help="Ground-truth CSV path (defaults to
../../data/<common-names>)")
ap.add_argument("--no-metrics", action="store_true", help="Skip F1 evaluation")
ap.add_argument("--limit", type=int, default=100, help="Max rows to print to
terminal (default: 100)")
ap.add_argument("--no-verbose", action="store_true", help="Silence pipeline
logs")
args = ap.parse_args(argv)

# Input
input_path = Path(args.input) if args.input else _resolve_default_input()
df = _read_input_csv(input_path)

# Run pipeline
start_time = time.time()
pipeline = RAGGPTPipeline(verbose=not args.no_verbose)
preds = pipeline.predict(df)
exec_time = time.time() - start_time

# Write CSV only if --results was passed; always print to terminal
if args.results:
    out_path = Path(args.results)
    out_path.parent.mkdir(parents=True, exist_ok=True)
    preds.to_csv(out_path, index=False, encoding="utf-8")
    print(f"[run_rag_gpt] Wrote predictions to {out_path}")

_print_predictions(preds, max_rows=args.limit)

# ------------------------------------------------------------------
# Metrics (F1) at the end
# ------------------------------------------------------------------
if args.no_metrics:
    return

if MetricsCalculator is None:
```

```
    print("[EVAL] metric_calculator.py no disponible en ../../evaluation;
omitiendo métricas.")
    return

truth_path = Path(args.truth) if args.truth else _resolve_default_truth()
if not truth_path or not truth_path.exists():
    print("[EVAL] Ground truth no encontrado (usa --truth <path> o coloca
ground_truth.csv/gold.csv/annotations.csv/labels.csv en ../../data).")
    return

try:
    gt = _read_truth_csv(truth_path)
except Exception as e:
    print(f"[EVAL] Error leyendo ground truth: {e}")
    return

# La clase suministrada compara por (note_id, concept_id).
calc = MetricsCalculator()
metrics = calc.calculate_metrics(predictions=preds, ground_truth=gt,
strategy_name="RAG+GPT")
report = calc.format_single_report(metrics, execution_time=exec_time,
strategy_name="RAG+GPT")
print(report)
```

if **name** == "**main**":
main()

--- Content of ./src/components/coding.py ---
"""Codificación SNOMED-CT usando RAG + GPT-4o (selección determinista + validación opcional)"""

import json
import os
import sys
from pathlib import Path
from typing import List, Dict, Tuple, Optional
from openai import OpenAI

Setup paths for absolute imports

COMPONENTS_DIR = Path(**file**).parent.resolve()
SRC_DIR = COMPONENTS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from components.rag import RAGRetriever

class SNOMEDCoder:
"""

Codificador de entidades médicas usando SNOMED-CT.

```
Cambios clave (solución robusta):
  - Selección determinista del mejor código basada en similitud (SapBERT/FAISS)
en Python.
  - El LLM (GPT-4o) pasa a ser VALIDACIÓN opcional, y NUNCA puede forzar
fallback si hay candidatos.
  - Umbrales y parámetros configurables por variables de entorno (sin parchear
ficheros).
  - Uso real de system prompt en llamadas al LLM.
  - Render seguro de plantillas (sin usar str.format en prompts con llaves
JSON).
"""

FALLBACK_CODE = "404684003"   # Clinical finding (genérico)
DEFAULT_ANATOMY = "12738006"  # Brain structure
PRESENCE_MAP = {
    "presente": "52101004",    # Present
    "ausente": "272519000",    # Absent
    "incierto": "261665006"    # Unknown
}

def __init__(self, rag_retriever: RAGRetriever, openai_client: OpenAI,
system_prompt: Optional[str] = None):
    self.rag = rag_retriever
    self.client = openai_client
    self.system_prompt = system_prompt or "You are a precise SNOMED-CT coding
assistant."

    # Cargar prompts desde la nueva ubicación
    from config import load_prompt
    self.prompt_config = load_prompt("coding")
    self.filter_config = load_prompt("filter")

    # Config runtime por ENV (robusto para optimización)
    self.cfg = {
        "TOP_K": int(os.getenv("RAG_TOP_K", "30")),
        # Stricter similarity by default to improve precision
        "THRESHOLD": float(os.getenv("RAG_THRESHOLD", "0.50")),
        "MAX_DISPLAY": int(os.getenv("RAG_MAX_DISPLAY", "12")),
        "QUERY_SUFFIX": os.getenv("RAG_QUERY_SUFFIX", "disorder finding"),
        "USE_LLM_VALIDATION": os.getenv("RAG_USE_LLM_VALIDATION",
"false").lower() == "true",
        "LLM_MODEL": os.getenv("RAG_LLM_MODEL", "gpt-4o"),
        "LLM_TEMPERATURE": float(os.getenv("RAG_LLM_TEMPERATURE", "0.0")),
        # NEW: disable generic fallback by default
        "ALLOW_FALLBACK": os.getenv("RAG_ALLOW_FALLBACK", "false").lower() ==
"true",
    }
```

```
# --------------------------
# API pública
# --------------------------
def code_entities(self, entities: List[Dict], verbose: bool = True) ->
List[Dict]:
    """Codifica entidades usando SNOMED-CT con selección determinista (+
validación opcional)."""
    if verbose:
        print(f"[CODING] Codificando {len(entities)} entidades...")

    coded_entities = []
    for entity in entities:
        codes = self.assign_codes(
            entity=entity['span_text'],
            location=entity.get('anatomical_location', 'No especificado'),
            presence=entity.get('presence', 'presente'),
            verbose=verbose
        )
        coded_entity = {**entity, **codes}
        coded_entities.append(coded_entity)

    return coded_entities

def assign_codes(self, entity: str, location: str, presence: str, verbose: bool
= False) -> dict:
    """
    Selección determinista:
      1) Recupera candidatos para ENTITY (double query) y ANATOMY.
      2) Elige top-1 por similitud (ya filtrado por THRESHOLD).
      3) (Opcional) Valida con LLM, restringido a los candidatos.
    """
    # 1) Recuperación de candidatos
    ent_results = self._retrieve_candidates(query=entity,
context_type="ENTITY", verbose=verbose)
    anat_results = self._retrieve_candidates(query=location,
context_type="ANATOMY", verbose=verbose) \
        if location and location != "No especificado" else []

    # 2) Selección determinista top-1 o fallback/default
    entity_code = self._pick_top_code(ent_results, self.cfg["THRESHOLD"])
    if entity_code is None and self.cfg["ALLOW_FALLBACK"]:
        entity_code = self.FALLBACK_CODE  # solo si se permite
    anatomy_code = self._pick_top_code(anat_results, self.cfg["THRESHOLD"]) or
self.DEFAULT_ANATOMY
    presence_code = self.PRESENCE_MAP.get(str(presence).lower(),
self.PRESENCE_MAP["presente"])

    # 3) Validación opcional con LLM (nunca puede forzar fallback si hay
candidatos)
```

```
    if self.cfg["USE_LLM_VALIDATION"]:
        if verbose:
            print(f"[CODING]   -> Validación con {self.cfg['LLM_MODEL']}
(restringido a candidatos)...")

        contexto_entity = self._format_context("ENTITY", entity, ent_results)
        contexto_anatomy = self._format_context("ANATOMY", location,
anat_results) if anat_results else "--- ANATOMY NOT SPECIFIED ---\n"
        valid_entity_list = [c for c, _, _ in ent_results]
        valid_anatomy_list = [c for c, _, _ in anat_results]

        # Render SEGURO del prompt (sin str.format)
        prompt = self._render_template(
            self.prompt_config["template"],
            {
                "entity": entity,
                "location": location,
                "presence": presence,
                "contexto_entity": contexto_entity,
                "contexto_anatomy": contexto_anatomy,
                "valid_entity_codes": json.dumps(valid_entity_list,
ensure_ascii=False),
                "valid_anatomy_codes": json.dumps(valid_anatomy_list,
ensure_ascii=False)
            }
        )

        try:
            response = self.client.chat.completions.create(
                model=self.cfg["LLM_MODEL"],
                messages=[
                    {"role": "system", "content": self.system_prompt},
                    {"role": "user", "content": prompt}
                ],
                temperature=self.cfg["LLM_TEMPERATURE"],
                response_format={"type": "json_object"}
            )
            result = json.loads(response.choices[0].message.content)

            # Post-procesado restrictivo (no permitir inventar/fallback si hay
candidatos)
            proposed_entity = str(result.get("entity_code", entity_code or
"")).strip()
            proposed_anatomy = str(result.get("anatomy_code",
anatomy_code)).strip()
            proposed_presence = str(result.get("presence_code",
presence_code)).strip()

            if valid_entity_list:
                if proposed_entity.isdigit() and proposed_entity in
```

```
valid_entity_list:
                    entity_code = proposed_entity
                else:
                    # Mantener determinista si LLM se sale del conjunto o
propone fallback
                    pass
            else:
                # No hay candidatos → aceptar lo que proponga si es numérico
                if proposed_entity.isdigit():
                    entity_code = proposed_entity

            if valid_anatomy_list:
                if proposed_anatomy.isdigit() and proposed_anatomy in
valid_anatomy_list:
                    anatomy_code = proposed_anatomy
            else:
                if proposed_anatomy.isdigit():
                    anatomy_code = proposed_anatomy

            if proposed_presence.isdigit():
                presence_code = proposed_presence

        except Exception as e:
            if verbose:
                print(f"[CODING]   [WARNING] Error en validación LLM: {e}")

    # Normalización final y logs (no forzar fallback por defecto)
    if entity_code is not None and not str(entity_code).isdigit():
        if verbose:
            print(f"[CODING]   [WARNING] entity_code no numérico:
'{entity_code}', descartando")
        entity_code = None

    if not str(anatomy_code).isdigit():
        if verbose:
            print(f"[CODING]   [WARNING] anatomy_code no numérico:
'{anatomy_code}', usando default")
        anatomy_code = self.DEFAULT_ANATOMY

    if not str(presence_code).isdigit():
        if verbose:
            print(f"[CODING]   [WARNING] presence_code no numérico:
'{presence_code}', usando default")
        presence_code = self.PRESENCE_MAP["presente"]

    if verbose:
        print(f"[CODING]   [OK] Códigos: entity={entity_code if entity_code
else '∅'}, anatomy={anatomy_code}, presence={presence_code}")

    return {
```

```
        "entity_code": str(entity_code) if entity_code else "",
        "anatomy_code": str(anatomy_code),
        "presence_code": str(presence_code)
    }

# -------------------------
# Helpers internos
# -------------------------
def _retrieve_candidates(self, query: str, context_type: str, verbose: bool) ->
List[Tuple[str, str, float]]:
    """
    Recupera, deduplica y filtra candidatos por THRESHOLD.
    Para ENTITY aplica double-query (query y query+suffix).
    """
    if not query or query == "No especificado":
        return []

    TOP_K = self.cfg["TOP_K"]
    THRESHOLD = self.cfg["THRESHOLD"]

    if context_type == "ENTITY":
        results_main = self.rag.retrieve(query, k=TOP_K)
        query_clinical = f"{query} {self.cfg['QUERY_SUFFIX']}".strip()
        results_clinical = self.rag.retrieve(query_clinical, k=TOP_K)

        combined = {}
        for concepto, narrativa, sim in (results_main + results_clinical):
            # Mantener la mayor similitud por concepto
            if concepto not in combined or sim > combined[concepto][1]:
                combined[concepto] = (narrativa, sim)

        results = [(c, n, s) for c, (n, s) in combined.items()]
    else:
        results = self.rag.retrieve(query, k=min(TOP_K, 15))

    # Filtrado y orden
    filtered = [(c, n, s) for c, n, s in results if s >= THRESHOLD]
    filtered.sort(key=lambda x: x[2], reverse=True)

    if verbose:
        if filtered:
            best_code, _, best_sim = filtered[0]
            print(f"[CODING]   -> RAG {context_type} [MULTI-Q]: {len(filtered)}
conceptos (mejor: {best_code}, dist: {best_sim:.3f})")
        else:
            print(f"[CODING]   -> RAG {context_type}: 0 resultados (dist <
{THRESHOLD})")

    return filtered
```

```
    def _pick_top_code(self, results: List[Tuple[str, str, float]], threshold:
    float) -> Optional[str]:
        """Elige top-1 si existe y supera threshold. Si no, None."""
        if not results:
            return None
        best_code, _, best_sim = results[0]
        return best_code if best_sim >= threshold and str(best_code).isdigit() else
    None

    def _format_context(self, context_type: str, query: str, results:
    List[Tuple[str, str, float]]) -> str:
        """Construye el bloque de contexto para el LLM (solo informativo)."""
        if not results:
            return f"--- {context_type} CODES for '{query}' ---\n--- NO CODES FOUND
    ---\n"

        MAX_DISPLAY = self.cfg["MAX_DISPLAY"]
        context = f"\n--- {context_type} CODES for '{query}' ---\n"
        for idx, (concepto, narrativa, sim) in enumerate(results[:MAX_DISPLAY], 1):
            context += f"OPCIÓN {idx} [SIM: {sim:.2f}]: CÓDIGO: {concepto} |
    {narrativa[:120]}\n"
        return context

    def _render_template(self, template: str, variables: Dict[str, str]) -> str:
        """
        Render muy simple para prompts con JSON: sustituye solo {clave} conocidas,
        sin interpretar el resto de llaves que forman parte del texto.
        """
        s = template
        for k, v in variables.items():
            s = s.replace("{" + k + "}", str(v))
        return s
```

--- Content of ./src/components/ner.py ---
"""Named Entity Recognition usando GPT-4o (con system prompt y offsets de caracteres)
Robusto:

- Limpieza centralizada del JSON (markdown/trailing commas).
- Extracción del PRIMER objeto JSON balanceando llaves (tolerante a texto extra).
- Fallback: parseo por-objeto dentro de "entities" (equilibra llaves y repara separadores).
- Saneado de caracteres de control y conversión segura de offsets.
  """

import json
import re
import sys
from pathlib import Path

from typing import List, Dict, Optional, Any
from openai import OpenAI

Setup paths for absolute imports

COMPONENTS_DIR = Path(**file**).parent.resolve()
SRC_DIR = COMPONENTS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

reutilizamos limpieza común

from utils.text import clean_json_response

class NERExtractor:
"""Extractor de entidades médicas usando GPT-4o"""

```
def __init__(self, client: OpenAI, prompt_config: Dict, model_config: Dict,
system_prompt: Optional[str] = None):
    self.client = client
    self.prompt_template = prompt_config['template']
    self.model_config = model_config
    self.system_prompt = system_prompt or "You are a precise clinical NER
extractor. Respond ONLY with valid JSON."

def extract_entities(self, texto: str) -> List[Dict]:
    """Extrae entidades médicas del texto usando GPT-4o"""
    print("[NER] Ejecutando extracción de entidades con GPT-4o...")

    # USAR render seguro (evita KeyError por llaves JSON en la plantilla)
    prompt = self._render(self.prompt_template, {"informe": texto})
    response = self._call_gpt4o(prompt)
    entities = self._parse_ner_response(response)

    print(f"[NER] Entidades detectadas: {len(entities)}")
    for i, ent in enumerate(entities[:3]):
        print(f"  - Span: \"{ent['span_text']}\"")
    if len(entities) > 3:
        print(f"  ... y {len(entities) - 3} más")

    return entities

def _call_gpt4o(self, prompt: str, max_retries: int = 3) -> str:
    """Llama a GPT-4o con manejo de errores"""
    for attempt in range(max_retries):
        try:
            response = self.client.chat.completions.create(
```

```
                model=self.model_config["model"],
                messages=[
                    {"role": "system", "content": self.system_prompt},
                    {"role": "user", "content": prompt}
                ],
                temperature=self.model_config.get("temperature", 0.1),
                max_tokens=self.model_config.get("max_tokens", 4000),
                response_format={"type": "json_object"}
            )
            return response.choices[0].message.content.strip()

        except Exception as e:
            print(f"[NER] Error en llamada GPT-4o (intento
{attempt+1}/{max_retries}): {e}")
            if attempt == max_retries - 1:
                return '{"entities": []}'

    return '{"entities": []}'

# ------------------------------
# Parsing y reparación de JSON
# ------------------------------
def _strip_control_chars(self, s: str) -> str:
    """Elimina caracteres de control no imprimibles que rompen JSON."""
    return ''.join(ch for ch in s if ch == '\t' or ch == '\n' or ch == '\r' or
ord(ch) >= 32)

def _extract_top_level_json(self, s: str) -> Optional[str]:
    """
    Devuelve el primer objeto JSON balanceando llaves, ignorando texto extra.
    Soporta respuestas con prólogo/epílogo.
    """
    # limpiar markdown + trailing commas + control chars
    s = clean_json_response(self._strip_control_chars(s))
    # quitar elipsis sueltas al final
    s = s.rstrip('.… \n\r\t')

    # buscar primera llave
    start = s.find('{')
    if start == -1:
        return None

    depth = 0
    in_str = False
    escape = False
    for i in range(start, len(s)):
        ch = s[i]
        if in_str:
            if escape:
                escape = False
```

```
            elif ch == '\\':
                escape = True
            elif ch == '"':
                in_str = False
        else:
            if ch == '"':
                in_str = True
            elif ch == '{':
                depth += 1
            elif ch == '}':
                depth -= 1
                if depth == 0:
                    return s[start:i+1]
    # si no cerró, intentar cerrar brutamente
    if depth > 0:
        return s[start:] + ('}' * depth)
    return None

def _repair_entities_array(self, text: str) -> str:
    """
    Repara separadores y cierra el array 'entities' si es evidente que quedó
abierto.
    """
    # arreglar objetos consecutivos sin coma dentro de arrays
    text = re.sub(r'}\s*{', '}, {', text)

    # si abre "entities": [ pero falta ']' antes de cerrar objeto raíz,
    # insertamos un ']' justo antes del último '}'.
    ent_open = text.find('"entities"')
    if ent_open != -1:
        arr_open = text.find('[', ent_open)
        if arr_open != -1:
            arr_close = text.find(']', arr_open)
            if arr_close == -1:
                # insertar ']' antes del último '}'
                last_brace = text.rfind('}')
                if last_brace != -1 and last_brace > arr_open:
                    text = text[:last_brace] + ']' + text[last_brace:]
    return text

def _iter_objects_in_entities(self, text: str) -> List[str]:
    """
    Extrae cada objeto del array "entities" balanceando llaves.
    Útil como Fallback cuando el JSON completo no parsea.
    """
    text = self._strip_control_chars(text)
    m = re.search(r'"entities"\s*:\s*\[', text)
    if not m:
        return []
    i = text.find('[', m.end() - 1)
```

```
        if i == -1:
            return []

        objs = []
        depth = 0
        in_str = False
        escape = False
        start_obj = None

        # recorremos desde el primer carácter dentro del array
        for j in range(i + 1, len(text)):
            ch = text[j]
            if in_str:
                if escape:
                    escape = False
                elif ch == '\\':
                    escape = True
                elif ch == '"':
                    in_str = False
            else:
                if ch == '"':
                    in_str = True
                elif ch == '{':
                    if depth == 0:
                        start_obj = j
                    depth += 1
                elif ch == '}':
                    depth -= 1
                    if depth == 0 and start_obj is not None:
                        objs.append(text[start_obj:j+1])
                        start_obj = None
                elif ch == ']':
                    if depth == 0:
                        break  # fin del array

        return objs

    def _safe_int(self, x):
        if x is None:
            return None
        try:
            if isinstance(x, str):
                x = x.strip()
                if not re.match(r'^-?\d+(\.0+)?$', x):
                    return None
            return int(float(x))
        except Exception:
            return None

    def _normalize_entity(self, finding: Dict[str, Any]) -> Optional[Dict[str,
```

```
Any]]:
    if not isinstance(finding, dict):
        return None
    core_entity = (finding.get("core_entity") or finding.get("entity") or
"").strip()
    full_span = (finding.get("full_span") or core_entity or "").strip()
    if not full_span:
        return None
    anatomical_location = (finding.get("anatomical_location") or "No
especificado") or "No especificado"
    presence = (finding.get("presence") or "presente") or "presente"
    value = finding.get("value", None)
    start_int = self._safe_int(finding.get("start", None))
    end_int = self._safe_int(finding.get("end", None))
    return {
        "span_text": core_entity if core_entity else full_span,
        "full_span": full_span,
        "anatomical_location": anatomical_location,
        "presence": presence,
        "value": value,
        "start": start_int,
        "end": end_int
    }

def _parse_ner_response(self, response: str) -> List[Dict]:
    """Parsea la respuesta JSON del NER y normaliza campos expected (tolerante
a errores)"""
    try:
        candidate = self._extract_top_level_json(response)
        if not candidate:
            raise ValueError("No se encontró objeto JSON en la respuesta")

        candidate = self._repair_entities_array(candidate)

        # Intento 1: parse completo
        try:
            data = json.loads(candidate)
            entities_raw = data.get("entities", [])
            if not isinstance(entities_raw, list):
                # buscar en cualquier clave una lista de dicts
                for v in data.values():
                    if isinstance(v, list) and any(isinstance(x, dict) for x in
v):
                        entities_raw = v
                        break
            entities: List[Dict[str, Any]] = []
            for finding in entities_raw:
                ent = self._normalize_entity(finding)
                if ent:
                    entities.append(ent)
```

```
            return entities

        except json.JSONDecodeError:
            # Intento 2: arreglos simples y reintento
            candidate2 = re.sub(r',(\s*[}\]])', r'\1', candidate)
            candidate2 = re.sub(r'}\s*{', '}, {', candidate2)
            try:
                data = json.loads(candidate2)
                entities_raw = data.get("entities", [])
                if not isinstance(entities_raw, list):
                    for v in data.values():
                        if isinstance(v, list) and any(isinstance(x, dict) for
x in v):
                            entities_raw = v
                            break
                entities = []
                for finding in entities_raw:
                    ent = self._normalize_entity(finding)
                    if ent:
                        entities.append(ent)
                return entities
            except json.JSONDecodeError:
                # Fallback definitivo: parsear cada objeto del array "entities"
por separado
                objs = self._iter_objects_in_entities(candidate2)
                entities = []
                for obj_str in objs:
                    obj_str = re.sub(r',(\s*[}\]])', r'\1', obj_str)
                    try:
                        finding = json.loads(obj_str)
                        ent = self._normalize_entity(finding)
                        if ent:
                            entities.append(ent)
                    except Exception:
                        # ignorar objetos irrecuperables
                        continue

                if entities:
                    return entities
                # si no se pudo recuperar nada:
                raise

    except Exception as e:
        print(f"[NER] Error parseando respuesta: {e}")
        print(f"[NER] Respuesta: {response[:500]}...")
        return []

def _render(self, template: str, variables: Dict[str, Any]) -> str:
    """
    Render muy simple: sustituye solo las claves provistas {clave} por su
```

```
valor,
    sin interpretar el resto de llaves del template (evita conflictos con
JSON).
    """
    s = template
    for k, v in variables.items():
        s = s.replace("{" + k + "}", str(v))
    return s
```

--- Content of ./src/components/rag/augmentation.py ---
"""
RAG Augmentation Module - Query expansion and bilingual hints
Enhances queries with EN↔ES translations and domain-specific expansions
"""

import os
import json
from typing import List, Dict, Iterable

class QueryAugmenter:
"""Gestiona expansión de queries y hints bilingües"""

```
def __init__(self, assets_dir: str):
    """
    Args:
        assets_dir: Directorio donde buscar bilingual_hints.json
    """
    self.assets_dir = assets_dir
    self.hints: Dict[str, Iterable[str]] = {}
    self._load_bilingual_hints()

def _load_bilingual_hints(self):
    """
    Carga mapeos EN→ES desde JSON opcional (assets/bilingual_hints.json).
    Si no existe, usa un conjunto por defecto.
    """
    default_hints = {
        "stroke": ["ictus", "accidente cerebrovascular"],
        "headache": ["cefalea", "dolor de cabeza"],
        "nausea": ["náusea"],
        "vomiting": ["vómito", "emesis"],
        "weakness": ["debilidad", "astenia"],
        "hemiparesis": ["hemiparesia"],
        "hemiplegia": ["hemiplejia"],
        "aphasia": ["afasia"],
        "dysarthria": ["disartria"],
        "atrial fibrillation": ["fibrilación auricular", "FA"],
        "thrombectomy": ["trombectomía", "terapia endovascular"],
```

```
        "tpa": ["alteplasa", "tPA", "rtPA"],
        "middle cerebral artery": ["arteria cerebral media", "ACM"],
        "mca": ["arteria cerebral media"],
        "occlusion": ["oclusión"],
        "recanalization": ["recanalización"],
        "ischemic penumbra": ["penumbra isquémica", "penumbra"],
        "hemorrhage": ["hemorragia", "hemorragia intracerebral", "hemorragia
intracraneal"],
        "infarct": ["infarto cerebral", "infarto isquémico"],
    }
    path = os.path.join(self.assets_dir, "bilingual_hints.json")
    try:
        if os.path.exists(path):
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)
            # normalizar a listas
            for k, v in data.items():
                if isinstance(v, str):
                    data[k.lower()] = [v]
                elif isinstance(v, list):
                    data[k.lower()] = list({str(x) for x in v if
str(x).strip()})
            self.hints = {**default_hints, **data}
            print(f"[AUGMENTATION] [OK] Hints bilingües cargados desde {path}
({len(self.hints)} entradas)")
        else:
            self.hints = default_hints
            print(f"[AUGMENTATION] [OK] Hints bilingües por defecto
({len(self.hints)} entradas)")
    except Exception as e:
        print(f"[AUGMENTATION] [WARNING] No se pudieron cargar hints desde
JSON: {e}")
        self.hints = default_hints

def expand_query_variants(self, query: str) -> List[str]:
    """
    Genera variantes de búsqueda:
    - original
    - lower
    - hints EN→ES si existen
    """
    variants = []
    if not isinstance(query, str):
        query = str(query)
    base = query.strip()
    if not base:
        return variants
    candidates = [base, base.lower()]
    key = base.lower()
    if key in self.hints:
```

```
            # puede ser lista
            for es in self.hints[key]:
                candidates.append(es)
        # deduplicación conservando orden
        seen = set()
        for c in candidates:
            c2 = c.strip()
            if c2 and c2 not in seen:
                seen.add(c2)
                variants.append(c2)
        return variants
```

--- Content of ./src/components/rag/generation.py ---
"""
RAG Generation Module - Optional LLM-based validation and re-ranking
(Currently a placeholder - LLM validation is handled in coding.py)
"""

from typing import List, Tuple, Dict

class LLMValidator:
"""Validador opcional basado en LLM (placeholder para extensiones futuras)"""

```
def __init__(self, client=None):
    """
    Args:
        client: Cliente OpenAI (opcional, para futuras extensiones)
    """
    self.client = client

def validate_candidates(
    self,
    query: str,
    candidates: List[Tuple[str, str, float]]
) -> List[Tuple[str, str, float]]:
    """
    Valida o re-rankea candidatos usando LLM.

    NOTA: Por ahora, retorna candidatos sin modificar.
    La validación real se hace en coding.py con restricción a candidatos.

    Args:
        query: Query original
        candidates: Lista de (concepto, narrativa, similitud)

    Returns:
        Lista de candidatos (sin modificar por ahora)
    """
```

```
    # Placeholder: la validación LLM real está en coding.py
    return candidates
```

--- Content of ./src/components/rag/rag.py ---
"""
RAG Orchestrator - Coordinates Retrieval + Augmentation + (optional) Generation
Main interface for the RAG subsystem
"""

import sys
from pathlib import Path
from typing import List, Tuple, Dict

Setup paths for absolute imports

RAG_DIR = Path(**file**).parent.resolve()
COMPONENTS_DIR = RAG_DIR.parent
SRC_DIR = COMPONENTS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from components.rag.retrieval import FAISSRetriever
from components.rag.augmentation import QueryAugmenter
from components.rag.generation import LLMValidator

class RAGRetriever:
"""
Sistema RAG completo que orquesta:
- Retrieval (FAISS/SapBERT)
- Augmentation (query expansion, hints bilingües)
- Generation (validación LLM opcional - actualmente en coding.py)
"""

```
def __init__(self, assets_dir: str):
    """
    Args:
        assets_dir: Directorio con índice FAISS y recursos
    """
    self.assets_dir = assets_dir

    # Componentes
    self.retriever = FAISSRetriever(assets_dir)
    self.augmenter = QueryAugmenter(assets_dir)
    self.validator = LLMValidator()  # placeholder

    print("[RAG] [OK] Sistema RAG inicializado (R+A+G)")
```

```
def retrieve(self, query: str, k: int = 5) -> List[Tuple[str, str, float]]:
    """
    Recupera conceptos similares usando búsqueda semántica con expansión
bilingüe.

    Pipeline:
    1. Augmentation: Genera variantes de la query (EN/ES, lowercase)
    2. Retrieval: Búsqueda multi-query con fusión por máximo
    3. (Generation): Validación/re-ranking opcional (placeholder)

    Args:
        query: Query de búsqueda
        k: Número de resultados a retornar

    Returns:
        Lista de tuplas (concepto, narrativa, similitud) ordenadas por
similitud
    """
    # 1. Augmentation: Generar variantes
    variants = self.augmenter.expand_query_variants(query)

    # 2. Retrieval: Multi-query con fusión
    results = self.retrieve_multi(variants, k=k)

    # 3. Generation: Validación opcional (por ahora no hace nada)
    validated = self.validator.validate_candidates(query, results)

    return validated

def retrieve_multi(self, queries: List[str], k: int = 5) -> List[Tuple[str,
str, float]]:
    """
    Ejecuta varias queries y fusiona resultados por máximo de similitud por
concepto.

    Args:
        queries: Lista de queries a ejecutar
        k: Número de resultados finales

    Returns:
        Lista fusionada de (concepto, narrativa, similitud)
    """
    if not queries:
        return []

    pool: Dict[str, Tuple[str, float]] = {}
    for q in queries:
        res = self.retriever.search(q, k)
        for concepto, narrativa, sim in res:
```

```
            # conservar la mejor similitud para cada concepto
            prev = pool.get(concepto)
            if (prev is None) or (sim > prev[1]):
                pool[concepto] = (narrativa, sim)

    fused = [(c, n, s) for c, (n, s) in pool.items()]
    fused.sort(key=lambda x: x[2], reverse=True)
    return fused[:k]
```

--- Content of ./src/components/rag/retrieval.py ---
"""
RAG Retrieval Module - FAISS/SapBERT semantic search
Handles FAISS index loading and embedding generation
"""

import os
import pickle
import faiss
from typing import List, Tuple
import torch
import numpy as np
from transformers import AutoTokenizer, AutoModel

--- MODELO CORRECTO (del README) ---

MODEL_NAME = 'cambridgeltl/SapBERT-from-PubMedBERT-fulltext'

class FAISSRetriever:
"""Sistema de recuperación semántica usando FAISS (SapBERT mean-pooling)"""

```
def __init__(self, assets_dir: str):
    """
    Args:
        assets_dir: Directorio con índice FAISS y archivos pickle
    """
    self.assets_dir = assets_dir
    self.faiss_index = None
    self.conceptos = []
    self.narrativas = []

    self.device = "cuda" if torch.cuda.is_available() else "cpu"
    self.tokenizer = None
    self.model = None

    self._load_model_and_tokenizer()
    self._load_index()
```

```
    self._load_ontology()

    # Validación de coherencia
    try:
        self._validate_index_coherence()
        print("[RETRIEVAL] [OK] Validación de índice superada")
    except Exception as e:
        print(f"[RETRIEVAL] [ERROR] {e}")
        raise

def _load_model_and_tokenizer(self):
    """Carga el modelo y tokenizador de HuggingFace (SapBERT-style)"""
    try:
        print(f"[RETRIEVAL] Cargando modelo y tokenizador: {MODEL_NAME}...")
        self.tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
        self.model = AutoModel.from_pretrained(MODEL_NAME).to(self.device)
        self.model.eval()
        print(f"[RETRIEVAL] [OK] Modelo cargado en {self.device}")
    except Exception as e:
        print(f"[RETRIEVAL] [ERROR] No se pudo cargar el modelo de HuggingFace:
{e}")

def _load_index(self):
    """Carga el índice FAISS pre-construido"""
    index_path = os.path.join(self.assets_dir, 'ontology.index')
    if not os.path.exists(index_path):
        print(f"[RETRIEVAL] [WARNING] Índice FAISS no encontrado en
{index_path}")
        return

    try:
        print(f"[RETRIEVAL] Cargando índice FAISS...")
        self.faiss_index = faiss.read_index(index_path)

        metadata_path = os.path.join(self.assets_dir, 'ontology_metadata.pkl')
        if os.path.exists(metadata_path):
            with open(metadata_path, 'rb') as f:
                metadata = pickle.load(f)
            print(f"[RETRIEVAL] [OK] Índice cargado: {metadata['n_concepts']}
conceptos (Modelo: {metadata.get('model_name', '??')})")

            # Validar que el índice se hizo con el mismo modelo
            if metadata.get('model_name') != MODEL_NAME:
                print(f"[RETRIEVAL] [WARNING] ¡El índice fue construido con un
modelo diferente ({metadata.get('model_name')})!")
                print(f"[RETRIEVAL] [WARNING] Por favor, borra los assets y re-
ejecuta build_index.py")

    except Exception as e:
        print(f"[RETRIEVAL] [ERROR] Error cargando índice: {e}")
```

```
def _load_ontology(self):
    """Carga conceptos y narrativas desde pickle"""
    concepts_path = os.path.join(self.assets_dir, 'ontology_concepts.pkl')
    narratives_path = os.path.join(self.assets_dir, 'ontology_narratives.pkl')

    try:
        with open(concepts_path, 'rb') as f:
            self.conceptos = pickle.load(f)
        with open(narratives_path, 'rb') as f:
            self.narrativas = pickle.load(f)
        print(f"[RETRIEVAL] [OK] Ontología cargada: {len(self.conceptos)}
conceptos")
    except Exception as e:
        print(f"[RETRIEVAL] [ERROR] Error cargando ontología: {e}")

def _get_query_embedding(self, query: str) -> np.ndarray:
    """Genera embedding normalizado (mean pooling con máscara)."""
    text = query if isinstance(query, str) else str(query)
    with torch.no_grad():
        toks = self.tokenizer.encode_plus(
            text,
            padding=True,
            max_length=64,   # ventana mayor para términos compuestos
            truncation=True,
            return_tensors="pt"
        )
        toks_on_device = {k: v.to(self.device) for k, v in toks.items()}

        outputs = self.model(**toks_on_device)
        last_hidden = outputs.last_hidden_state            # (B, T, H)
        mask = toks_on_device["attention_mask"].unsqueeze(-1)  # (B, T, 1)

        # mean pooling sobre tokens válidos
        sum_vec = (last_hidden * mask).sum(dim=1)          # (B, H)
        len_vec = mask.sum(dim=1).clamp(min=1)             # (B, 1)
        mean_vec = sum_vec / len_vec

        emb = mean_vec.cpu().numpy()
        norm = np.linalg.norm(emb, axis=1, keepdims=True)
        normalized_emb = emb / np.clip(norm, 1e-12, None)

        return normalized_emb.astype("float32")

def search(self, query: str, k: int) -> List[Tuple[str, str, float]]:
    """
    Búsqueda FAISS para una sola query.

    Returns:
        Lista de tuplas (concepto, narrativa, similitud)
```

```
    """
    if self.faiss_index is None or self.model is None:
        print("[RETRIEVAL] [WARNING] Sistema RAG no disponible. Faltan índice o
modelo.")
        return []

    try:
        q_emb = self._get_query_embedding(query)
        k = int(k)
        distances, indices = self.faiss_index.search(q_emb, k)

        results = []
        for i, idx in enumerate(indices[0]):
            if idx < 0:
                continue
            if idx < len(self.conceptos):
                concepto = self.conceptos[idx]
                narrativa = self.narrativas[idx]
                similarity = float(distances[0][i])
                if str(concepto).isdigit():
                    results.append((concepto, narrativa, similarity))
        return results
    except Exception as e:
        print(f"[RETRIEVAL] [ERROR] Error en búsqueda: {e}")
        import traceback
        traceback.print_exc()
        return []

def _validate_index_coherence(self):
    """Valida que el índice sea IP (coseno), tamaños coincidan y el modelo sea
el esperado."""
    if self.faiss_index is None:
        raise RuntimeError("Índice FAISS no cargado.")

    # (a) métrica = inner product (cosine con embeddings normalizados)
    try:
        metric = self.faiss_index.metric_type
    except Exception:
        metric = None

    if metric != faiss.METRIC_INNER_PRODUCT:
        raise RuntimeError(
            f"Índice FAISS con métrica no soportada ({metric}). "
            f"Reconstruye el índice con IndexFlatIP y embeddings L2-
normalizados."
        )

    # (b) tamaños deben coincidir
    if self.faiss_index.ntotal != len(self.conceptos) or len(self.conceptos) !=
len(self.narrativas):
```

```
            raise RuntimeError(
                f"Inconsistencia: index.ntotal={self.faiss_index.ntotal}, "
                f"conceptos={len(self.conceptos)}, narrativas=
  {len(self.narrativas)}."
            )

        # (c) modelo del índice debe coincidir
        metadata_path = os.path.join(self.assets_dir, 'ontology_metadata.pkl')
        if os.path.exists(metadata_path):
            with open(metadata_path, 'rb') as f:
                metadata = pickle.load(f)
            idx_model = metadata.get('model_name')
            if idx_model != MODEL_NAME:
                raise RuntimeError(
                    f"El índice se construyó con '{idx_model}' y el runtime usa
  '{MODEL_NAME}'. "
                    f"Reconstruye el índice con el mismo modelo."
                )
```

--- Content of ./src/components/rag/**init**.py ---

This file is required for Python to recognize this directory as a package

from .rag import RAGRetriever

all = ['RAGRetriever']

--- Content of ./src/components/**init**.py ---

Marks components as a package

--- Content of ./src/config.py ---
"""
Configuration management for the RAG+GPT pipeline.

- Loads environment from .env (if present)
- Centralizes prompt loading
- Provides OpenAI client, model config, and assets dir
  """

from **future** import annotations

import json
import os
from pathlib import Path

from typing import Dict
from openai import OpenAI

Resolve project root and .env

SRC_DIR = Path(**file**).parent.resolve()
PROJECT_ROOT = SRC_DIR.parent # repo root expected at: project/
ENV_PATH = PROJECT_ROOT / ".env"

Optional: python-dotenv

try:
from dotenv import load_dotenv # type: ignore
if ENV_PATH.exists():
load_dotenv(ENV_PATH)
print(f"[CONFIG] Loaded environment from: {ENV_PATH}")
else:
print(f"[CONFIG] No .env found at {ENV_PATH}; using system env")
except Exception:
print("[CONFIG] python-dotenv not available; using system env")

Evaluation offsets policy

EVAL_OFFSETS = {
"base": int(os.getenv("EVAL_OFFSET_BASE", "0")), # 0-based or 1-based
Most clinical benchmarks expect end to be EXCLUSIVE. Flip default to false.

"end_inclusive": os.getenv("EVAL_END_INCLUSIVE", "false").lower() == "true",
}

(Optional) tiny log helps verify at runtime

print(f"[CONFIG] EVAL_OFFSETS base={EVAL_OFFSETS['base']} end_inclusive=
{EVAL_OFFSETS['end_inclusive']}")

Prompt utilities

---

def load_prompt(prompt_name: str, prompts_dir: str | Path | None = None) -> Dict:
"""
Load a prompt JSON by name from src/prompts.

```
Args:
    prompt_name: name without .json
    prompts_dir: override directory

Returns:
    Dict
"""
if prompts_dir is None:
    prompts_dir = SRC_DIR / "prompts"
prompt_path = Path(prompts_dir) / f"{prompt_name}.json"
if not prompt_path.exists():
    raise FileNotFoundError(f"Prompt not found: {prompt_path}")
with open(prompt_path, "r", encoding="utf-8") as f:
    return json.load(f)
```

OpenAI client and model cfg

```
def setup_openai_client() -> OpenAI:
"""
Initialize OpenAI client using OPENAI_API_KEY (preferred),
with a fallback to 'api_keys' file containing a line 'chatGPT='.
"""
api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
api_file = PROJECT_ROOT / "api_keys"
try:
with open(api_file, "r", encoding="utf-8") as f:
for line in f:
if line.startswith("chatGPT="):
api_key = line.split("=", 1)[1].strip()
break
except Exception as e:
print(f"[CONFIG] Could not read {api_file}: {e}")
if not api_key:
print("[CONFIG] WARNING: OPENAI_API_KEY not set; using placeholder.")
api_key = "YOUR_API_KEY_HERE"
return OpenAI(api_key=api_key)

def get_model_config() -> Dict:
"""Config for GPT-4o usage."""
return {
"model": os.getenv("NER_LLM_MODEL", "gpt-4o"),
"temperature": float(os.getenv("NER_TEMPERATURE", "0.1")),
"max_tokens": int(os.getenv("NER_MAX_TOKENS", "4000")),
"top_p": float(os.getenv("NER_TOP_P", "0.9")),
}
```

Assets (FAISS index, ontology artifacts)

def get_assets_dir() -> Path:
"""Return absolute path to assets/ontology/"""
return PROJECT_ROOT / "assets" / "ontology"

--- Content of ./src/indexer/build_index.py ---
#!/usr/bin/env python3
"""
Offline Index Builder for RAG Strategy (Ontology Pre-processor)
Robusto:

- Extrae 'formas superficiales' (término preferido + sinónimos) de la narrativa.
- Usa mean pooling con ventana amplia (max_length=128, padding=True).
- Mantiene la narrativa original para mostrar contexto, pero embebe los textos depurados.
  """

import pandas as pd
import numpy as np
import pickle
import os
import sys
from pathlib import Path
import faiss
from datetime import datetime
import torch
from tqdm.auto import tqdm
from transformers import AutoTokenizer, AutoModel
import re

--- PATH SETUP ---

SCRIPT_DIR = Path(**file**).parent.resolve()
SRC_DIR = SCRIPT_DIR.parent
PROJECT_ROOT = SRC_DIR.parent

Assets directory (nueva ubicación)

ASSETS_DIR = PROJECT_ROOT / "assets" / "ontology"
ASSETS_DIR.mkdir(parents=True, exist_ok=True)

Output paths

INDEX_PATH = ASSETS_DIR / 'ontology.index'
CONCEPTS_PATH = ASSETS_DIR / 'ontology_concepts.pkl'
NARRATIVES_PATH = ASSETS_DIR / 'ontology_narratives.pkl'
METADATA_PATH = ASSETS_DIR / 'ontology_metadata.pkl'

--- MODELO CORRECTO (del README) ---

MODEL_NAME = 'cambridgeltl/SapBERT-from-PubMedBERT-fulltext'

==========

Limpieza: formas superficiales

==========

_PREF_RE = re.compile(
r"tiene\s+t[eé]rmino\s+preferido\s+([^0-9]+?)(?=\s+\d{3,}|\s+tiene|\s+se define|$)",
re.IGNORECASE
)
_SYN_RE = re.compile(
r"tiene\s+sin[oó]nimo\s+([^0-9]+?)(?=\s+\d{3,}|\s+tiene|\s+se define|$)",
re.IGNORECASE
)

def extract_surface_forms(narr: str, max_terms: int = 24) -> str:
"""
Extrae 'término preferido' y todos los 'sinónimo' de la narrativa en español.
Devuelve una cadena corta adecuada para SapBERT.
"""
if not isinstance(narr, str) or not narr.strip():
return ""

```
terms = []

m = _PREF_RE.search(narr)
if m:
    terms.append(m.group(1).strip())

syns = _SYN_RE.findall(narr)
terms.extend([s.strip() for s in syns])

# limpieza de colas y deduplicación case-insensitive
clean = []
seen = set()
for t in terms:
    t = re.sub(r"[.;,:]+$", "", t).strip()
    tl = t.lower()
    if tl and tl not in seen:
        seen.add(tl)
```

```
        clean.append(t)

# fallback si no encontramos nada útil
if not clean:
    # usa la narrativa pero sin el ruido más obvio (trozos 'se define como:')
    narr_short = re.split(r"\bse\s+define\s+como\b", narr, flags=re.IGNORECASE)
[0]
    return narr_short.strip()[:512]

return " [SEP] ".join(clean[:max_terms])
```

def load_ontology_csv():
"""Carga la ontología híbrida"""
print("\n" + "="*80)
print("STEP 1: Loading Ontology Data")
print("="*80)

```
hybrid_path = PROJECT_ROOT / 'ontology' / 'hybrid_ontology.csv'

if hybrid_path.exists():
    print(f"[INFO] Loading HYBRID ONTOLOGY from: {hybrid_path}")
    df = pd.read_csv(hybrid_path)
    print(f"[SUCCESS] Loaded {len(df)} concepts")
    return df

raise FileNotFoundError(f"Could not find ontology CSV at: {hybrid_path}")
```

def generate_embeddings(narratives, model_name=MODEL_NAME, batch_size=64):
"""
Genera embeddings con mean pooling (más robusto que [CLS] para SapBERT).
Normaliza L2 para IndexFlatIP (cosine).
Usa textos depurados (formas superficiales).
"""
print("\n" + "="*80)
print("STEP 2: Generating Embeddings (SapBERT mean pooling)")
print("="*80)

```
print(f"[INFO] Loading HuggingFace model: {model_name}")

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"[INFO] Using device: {device}")

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name).to(device)
model.eval()
```

```
    # ---- depurar narrativas ----
    cleaned = [extract_surface_forms(x) for x in narratives]

    print(f"[INFO] Encoding {len(cleaned)} narratives in batches of {batch_size}")

    all_embs_list = []

    with torch.no_grad():
        for i in tqdm(np.arange(0, len(cleaned), batch_size)):
            batch = cleaned[i:i+batch_size]

            toks = tokenizer.batch_encode_plus(
                batch,
                padding=True,          # padding dinámico
                max_length=128,        # ventana amplia para cubrir sinónimos
                truncation=True,
                return_tensors="pt"
            )
            toks_on_device = {k: v.to(device) for k, v in toks.items()}

            outputs = model(**toks_on_device)
            last_hidden = outputs.last_hidden_state           # (B, T, H)
            mask = toks_on_device["attention_mask"].unsqueeze(-1)  # (B, T, 1)

            sum_vec = (last_hidden * mask).sum(dim=1)         # (B, H)
            len_vec = mask.sum(dim=1).clamp(min=1)            # (B, 1)
            mean_vec = sum_vec / len_vec                      # (B, H)

            all_embs_list.append(mean_vec.cpu().numpy())

    all_embs = np.concatenate(all_embs_list, axis=0)
    print(f"[SUCCESS] Generated embeddings with shape: {all_embs.shape}")

    print("[INFO] Normalizing embeddings for Cosine Similarity (L2
    normalization)...")
    norms = np.linalg.norm(all_embs, axis=1, keepdims=True)
    normalized_embs = all_embs / np.clip(norms, 1e-12, None)

    print("[SUCCESS] Embeddings normalized.")
    return normalized_embs, all_embs.shape[1]
```

def build_faiss_index(embeddings):
"""Construye el índice FAISS (IndexFlatIP para Coseno)"""
print("\n" + "="*80)
print("STEP 3: Building Faiss Index")
print("="*80)

```
dimension = embeddings.shape[1]
n_concepts = embeddings.shape[0]

print(f"[INFO] Creating IndexFlatIP (cosine similarity) with dimension=
{dimension}")
index = faiss.IndexFlatIP(dimension)

print(f"[INFO] Adding {n_concepts} vectors to index...")
index.add(embeddings.astype('float32'))

print(f"[SUCCESS] Faiss index built with {index.ntotal} vectors")
return index
```

def save_artifacts(index, concepts, narratives, embedding_dim):
"""Guarda los artefactos en disco"""
print("\n" + "="*80)
print("STEP 4: Saving Artifacts")
print("="*80)

```
# Guardar índice
print(f"[INFO] Saving Faiss index to: {INDEX_PATH}")
faiss.write_index(index, str(INDEX_PATH))
print(f"[SUCCESS] Index saved ({os.path.getsize(INDEX_PATH) / 1024 / 1024:.2f}
MB)")

# Guardar listas (narrativas originales para mostrar contexto)
with open(CONCEPTS_PATH, 'wb') as f:
    pickle.dump(concepts, f)
print(f"[SUCCESS] Concepts saved ({len(concepts)} items)")

with open(NARRATIVES_PATH, 'wb') as f:
    pickle.dump(narratives, f)
print(f"[SUCCESS] Narratives saved ({len(narratives)} items)")

# Guardar metadata
metadata = {
    'n_concepts': len(concepts),
    'embedding_dim': embedding_dim,
    'model_name': MODEL_NAME,
    'created_at': datetime.now().isoformat(),
    'index_type': 'IndexFlatIP'
}

with open(METADATA_PATH, 'wb') as f:
    pickle.dump(metadata, f)
print(f"[SUCCESS] Metadata saved")
```

def main():
"""Flujo principal de ejecución"""
print("\n" + "="*80)
print("RAG INDEX BUILDER - Offline Pre-computation")
print("="*80)

```
import time
start_time = time.time()

try:
    # 1. Cargar
    df_ontology = load_ontology_csv()
    concepts = df_ontology['concepto'].tolist()
    narratives = df_ontology['narrativa'].tolist()

    # 2. Generar Embeddings (sobre formas superficiales)
    embeddings, dim = generate_embeddings(narratives)

    # 3. Construir Índice
    faiss_index = build_faiss_index(embeddings)

    # 4. Guardar
    save_artifacts(faiss_index, concepts, narratives, dim)

    elapsed_time = time.time() - start_time
    print("\n" + "="*80)
    print("[SUCCESS] INDEX BUILD COMPLETED SUCCESSFULLY")
    print(f"[TIME]  Total time: {elapsed_time:.2f} seconds
({elapsed_time/60:.2f} minutes)")
    print("="*80 + "\n")

except Exception as e:
    print("\n" + "="*80)
    print("[ERROR] ERROR BUILDING INDEX")
    print("="*80)
    print(f"\n{type(e).__name__}: {e}")
    import traceback
    traceback.print_exc()
    sys.exit(1)
```

if **name** == "**main**":
main()

--- Content of ./src/indexer/**init**.py ---

Marks indexer as a package

--- Content of ./src/pipeline.py ---
"""
RAG+GPT Pipeline - Orquestador principal
Implementa el pipeline completo de NER -> RAG -> Coding
"""

import os
import sys
import pandas as pd
from typing import List, Dict, Tuple
from pathlib import Path

Setup paths for absolute imports

SRC_DIR = Path(**file**).parent.resolve()
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from components.ner import NERExtractor
from components.rag import RAGRetriever
from components.coding import SNOMEDCoder
from config import (
load_prompt,
setup_openai_client,
get_model_config,
get_assets_dir,
EVAL_OFFSETS,
)
from utils.text import (
find_span_in_text,
find_all_spans_in_text,
find_exact_span,
find_exact_span_near,
find_first_case_insensitive,
tighten_span_boundaries,
)

class RAGGPTPipeline:
"""
Pipeline modular para extracción y codificación de entidades médicas

```
Arquitectura:
    1. NER: Extracción de entidades (GPT-4o) con spans VERBATIM + offsets
    2. RAG: Recuperación de conceptos SNOMED (FAISS)
    3. Coding: Codificación SNOMED-CT (selección determinista + validación
opcional)
```

```
    4. Span Matching: Uso de offsets del NER con corrección exacta (sin
expansión masiva)
"""

def __init__(self, verbose: bool = True):
    """
    Inicializa el pipeline completo

    Args:
        verbose: Si True, imprime logs detallados
    """
    self.verbose = verbose

    if verbose:
        print("=" * 80)
        print("RAG+GPT Pipeline - Inicialización")
        print("=" * 80)

    # 1. Configuración
    self.client = setup_openai_client()
    self.model_config = get_model_config()
    assets_dir = get_assets_dir()

    # Flag de recorte de bordes (activado por defecto)
    self.span_tighten = os.getenv("RAG_SPAN_TIGHTEN", "true").lower() == "true"

    # 2. Cargar prompts
    ner_prompt = load_prompt("ner")
    coding_prompt = load_prompt("coding")
    system_prompt_data = load_prompt("system")
    system_prompt = system_prompt_data["content"]

    # 3. Inicializar componentes
    self.rag = RAGRetriever(str(assets_dir))
    self.ner = NERExtractor(self.client, ner_prompt, self.model_config,
system_prompt=system_prompt)
    self.coder = SNOMEDCoder(self.rag, self.client,
system_prompt=system_prompt)

    if verbose:
        print("[OK] Pipeline inicializado correctamente")
        print("=" * 80)

# ------------------------------------------------------------------------
-----
# Normalización con mapeo de offsets (CRLF -> LF) SIN desalinear respecto al
original
# ------------------------------------------------------------------------
-----
def _normalize_text_with_mapping(self, original_text: str) -> Tuple[str,
```

```
List[int]]:
    """
    Devuelve:
      - texto_normalizado: reemplaza \r\n y \r por \n
      - mapping: lista de longitud len(texto_normalizado) que, para cada índice
i
                 del texto normalizado, da el índice correspondiente en el
texto original.
    También añadimos un "pivote" final implícito (len(original)) para calcular
'end'.
    """
    norm_chars: List[str] = []
    mapping: List[int] = []

    i = 0
    n = len(original_text)
    while i < n:
        ch = original_text[i]
        if ch == "\r":
            # Caso CRLF -> un solo '\n'
            if i + 1 < n and original_text[i + 1] == "\n":
                norm_chars.append("\n")
                mapping.append(i)  # mapeamos el '\n' normalizado al inicio del
par \r\n
                i += 2
            else:
                # CR suelto -> normalizamos a '\n'
                norm_chars.append("\n")
                mapping.append(i)
                i += 1
        else:
            norm_chars.append(ch)
            mapping.append(i)
            i += 1

    normalized = "".join(norm_chars)
    return normalized, mapping

def _map_norm_span_to_original(self, s_norm: int, e_norm: int, mapping:
List[int], orig_len: int) -> Tuple[int, int]:
    """
    Convierte un span [s_norm, e_norm) del texto normalizado a offsets del
texto original.
    Regla: start_orig = mapping[s_norm] ; end_orig = mapping[e_norm-1] + 1 (si
e_norm > 0)
    """
    if not mapping:
        return s_norm, e_norm  # sin normalización
    # start
    if s_norm < 0:
```

```
        s_norm = 0
    if s_norm >= len(mapping):
        start_orig = orig_len
    else:
        start_orig = mapping[s_norm]

    # end
    if e_norm <= 0:
        end_orig = 0
    elif e_norm - 1 >= len(mapping):
        end_orig = orig_len
    else:
        end_orig = mapping[e_norm - 1] + 1

    # sanity
    if end_orig < start_orig:
        end_orig = start_orig
    return start_orig, end_orig

# ------------------------------------------------------------------------
-----

def _chunk_text(self, text: str):
    """Divide texto en chunks con overlap y devuelve (chunk_text,
base_offset)."""
    chunk_size = 3000
    overlap = 300

    if len(text) <= chunk_size:
        return [(text, 0)]

    chunks = []
    start = 0
    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunks.append((text[start:end], start))
        if end >= len(text):
            break
        start = end - overlap

    if self.verbose:
        print(f"[CHUNKING] {len(chunks)} chunks (size={chunk_size}, overlap=
{overlap})")

    return chunks

def _deduplicate_entities(self, entities: List[Dict]) -> List[Dict]:
    """
    Elimina duplicados SOLO si son la misma ocurrencia (mismo start/end).
    """
```

```
    seen = set()
    unique: List[Dict] = []

    for e in entities:
        start = e.get("start")
        end = e.get("end")
        key = (
            e.get("full_span", e["span_text"]),
            e.get("anatomical_location", ""),
            e.get("presence", ""),
            start if isinstance(start, int) else None,
            end if isinstance(end, int) else None,
        )
        if key not in seen:
            seen.add(key)
            unique.append(e)

    if self.verbose and len(entities) > len(unique):
        print(f"[DEDUP] {len(entities)} -> {len(unique)} entidades (por
ocurrencia)")

    return unique

# Helper para añadir entidad localizada con "tighten"
def _append_located(self, located_entities: List[Dict], base_entity: Dict,
text: str, s: int, e: int):
    if self.span_tighten:
        s, e = tighten_span_boundaries(text, s, e)
    ent = dict(base_entity)
    ent["start"] = s
    ent["end"] = e
    ent["span_text_real"] = text[s:e]
    located_entities.append(ent)

def process_note(self, text: str, note_id: int = None) -> List[Dict]:
    """
    Procesa una nota médica completa

    Pipeline:
        original_text -> normalización+mapa -> Chunking -> NER -> Dedup ->
RAG+Coding -> Span Matching
        (y finalmente mapeo inverso de offsets al texto original)
    """
    if self.verbose and note_id:
        print(f"\n{'=' * 80}")
        print(f"Procesando nota {note_id}")
        print(f"{'=' * 80}")

    # --- Normalización con mapeo (CRLF/LF) ---
    original_text = text
```

```
    text_norm, mapping = self._normalize_text_with_mapping(original_text)
    text = text_norm  # a partir de aquí trabajamos SIEMPRE sobre el
normalizado

    # Chunking SIEMPRE (antes estaba dentro de un if)
    chunks = self._chunk_text(text)

    all_entities: List[Dict] = []
    for i, (chunk, base) in enumerate(chunks):
        if self.verbose and len(chunks) > 1:
            print(f"\n[NER] Procesando chunk {i + 1}/{len(chunks)} (base=
{base})...")

        chunk_entities = self.ner.extract_entities(chunk)

        # Rebasa offsets relativos del chunk a offsets globales del documento
normalizado
        for e in chunk_entities:
            if isinstance(e.get("start"), int):
                e["start"] += base
            if isinstance(e.get("end"), int):
                e["end"] += base

        all_entities.extend(chunk_entities)

    if not all_entities:
        if self.verbose:
            print("[WARNING] No se detectaron entidades")
        return []

    # Paso 1.5: Deduplicación - por ocurrencia (mantiene instancias con
distintos offsets)
    entities = self._deduplicate_entities(all_entities)

    # Paso 2: RAG + Coding - Codificar entidades (determinista + validación
opcional)
    coded_entities = self.coder.code_entities(entities, verbose=self.verbose)

    # Paso 3: Span Matching - Localización estricta sobre el TEXTO NORMALIZADO
    final_entities_norm = self._locate_spans(coded_entities, text)

    # Paso 4: MAPEO INVERSO de offsets al TEXTO ORIGINAL (clave para no romper
el benchmark)
    final_entities: List[Dict] = []
    for ent in final_entities_norm:
        s_norm = ent["start"]
        e_norm = ent["end"]
        s_orig, e_orig = self._map_norm_span_to_original(s_norm, e_norm,
mapping, len(original_text))
```

```
        ent_out = dict(ent)
        ent_out["start"] = s_orig
        ent_out["end"] = e_orig
        ent_out["span_text_real"] = original_text[s_orig:e_orig]  # texto
original exacto
        final_entities.append(ent_out)

    if self.verbose:
        print(f"\n[OK] Procesamiento completado: {len(final_entities)}
entidades")

    return final_entities

def _locate_spans(self, entities: List[Dict], text: str) -> List[Dict]:
    """
    Localiza spans con política ESTRICTA orientada a 'exact match' y recorte
opcional:
      1) Si la entidad trae start/end válidos y el snippet coincide EXACTO con
full_span → aceptar (y recortar bordes).
      2) Si hay offsets pero el snippet no cuadra, intentamos corregir cerca
del offset con búsqueda EXACTA.
      3) Si no hay offsets, buscamos UNA única ocurrencia EXACTA en todo el
texto.
      4) Como último recurso, usamos coincidencia case-insensitive (UNA sola
ocurrencia).
      5) NO expandimos a todas las ocurrencias (evita FPs y offsets no
anotados).
    """
    located_entities: List[Dict] = []

    for entity in entities:
        core_entity = entity["span_text"]
        full_span = (entity.get("full_span") or core_entity) or core_entity

        start = entity.get("start")
        end = entity.get("end")

        # (1) Usar offsets proporcionados si son válidos y EXACTOS (case-
sensitive)
        if isinstance(start, int) and isinstance(end, int) and 0 <= start < end
<= len(text):
            snippet = text[start:end]
            if snippet == full_span:
                self._append_located(located_entities, entity, text, start,
end)
                continue
            # (2) Corregir cerca del offset con búsqueda EXACTA
            nearby = find_exact_span_near(full_span, text, approx_start=start,
window=80)
            if nearby:
```

```
                s2, e2 = nearby
                self._append_located(located_entities, entity, text, s2, e2)
                continue
            # Intentar con el core_entity si el full_span falla
            nearby_core = find_exact_span_near(core_entity, text,
approx_start=start, window=80)
            if nearby_core:
                s3, e3 = nearby_core
                self._append_located(located_entities, entity, text, s3, e3)
                continue
            # Últimos recursos: global exacta
            global_match = find_exact_span(full_span, text)
            if global_match:
                s4, e4 = global_match
                self._append_located(located_entities, entity, text, s4, e4)
                continue
            # Case-insensitive global (una única ocurrencia)
            ci = find_first_case_insensitive(full_span, text)
            if ci:
                s5, e5 = ci
                self._append_located(located_entities, entity, text, s5, e5)
                continue

            if self.verbose:
                print(f"[SPAN] Descarta entidad por offsets no fiables y sin
match exacto: '{full_span[:40]}'")
            continue

        # (3) Sin offsets → buscar UNA ocurrencia exacta global
        exact_global = find_exact_span(full_span, text)
        if exact_global:
            s, e = exact_global
            self._append_located(located_entities, entity, text, s, e)
            continue

        # (4) Último recurso: case-insensitive UNA ocurrencia
        ci_global = find_first_case_insensitive(full_span, text)
        if ci_global:
            s2, e2 = ci_global
            self._append_located(located_entities, entity, text, s2, e2)
            continue

        # (5) No forzar regex flexible ni expansión
        if self.verbose:
            print(f"[SPAN] Sin offsets y sin match exacto: descarta
'{full_span[:40]}'")

    return located_entities

def predict(self, notes_df: pd.DataFrame) -> pd.DataFrame:
```

```
    """
    Procesa múltiples notas y genera predicciones

    Args:
        notes_df: DataFrame con columnas 'note_id' y 'text'

    Returns:
        DataFrame con predicciones en formato de benchmark
    """
    print(f"[Pipeline] Procesando {len(notes_df)} notas...")

    predictions = []

    for idx, row in notes_df.iterrows():
        note_id = row["note_id"]
        text = row["text"]

        # Procesar nota
        entities = self.process_note(text, note_id)

        # Convertir a formato de predicción con política de offsets del
benchmark
        for entity in entities:
            # ---- FAST WIN: filtrar entidades negadas/inciertas y fallback
genérico ----
            pres_text = str(entity.get("presence", "")).strip().lower()
            if pres_text in {"ausente", "incierto"}:
                continue  # sube precisión

            concept_id = str(entity.get("entity_code", "")).strip()
            if not concept_id or concept_id == "404684003":
                continue  # evita fallback genérico

            start_out = int(entity["start"])
            end_out = int(entity["end"])

            # Ajuste de inclusividad del 'end'
            if EVAL_OFFSETS.get("end_inclusive", False):
                # end es exclusivo internamente -> inclusivo para el benchmark
                end_out = max(0, end_out - 1)

            # Base 1 vs base 0
            if int(EVAL_OFFSETS.get("base", 0)) == 1:
                start_out += 1
                end_out += 1

            predictions.append({
                "note_id": note_id,
                "start": start_out,
                "end": end_out,
```

```
                "concept_id": concept_id,
                "span_text": entity.get("span_text_real", entity["span_text"]),
                "confidence": 0.85,
                "entity_description": entity.get("entity_description", ""),
                "anatomy_code": entity.get("anatomy_code", ""),
                "presence_code": entity.get("presence_code", ""),
                "llm_used": "GPT-4o",
            })

    print(f"[Pipeline] [OK] Completado: {len(predictions)} predicciones
generadas")

    return pd.DataFrame(predictions)
```

--- Content of ./src/prompts/coding.json ---
{
"name": "Codificación SNOMED-CT",
"version": "11.0 - Deterministic First, LLM-Validation Restricted",
"description": "Selección determinista (SapBERT/FAISS) + validación opcional con LLM restringida a candidatos",
"language": "es",
"template": "Eres un codificador SNOMED-CT muy estricto. SOLO puedes elegir códigos de las listas de candidatos proporcionadas. No inventes códigos. No uses el código fallback si existen candidatos.\n\n🎯 ENTITY TO CODE: "{entity}"\n📍 ANATOMICAL LOCATION: "{location}"\n☑️ PRESENCE STATUS: "{presence}"\n\n{contexto_entity}\n\n{contexto_anatomy}\n\nREGLAS OBLIGATORIAS:\n1) entity_code:\n - SIEMPRE elige uno de los códigos listados en las OPCIONES del bloque ENTITY.\n - Si el bloque ENTITY muestra "--- NO CODES FOUND ---", entonces y solo entonces puedes usar "404684003".\n - Si hay múltiples opciones, elige la que sea más específica para la entidad dada (pero SIEMPRE dentro de la lista).\n\n2) anatomy_code:\n - Si Location = "No especificado" o no hay candidatos en ANATOMY → usa "12738006" (Brain structure).\n - En caso contrario, elige SIEMPRE uno de los códigos listados en ANATOMY (el más apropiado).\n\n3) presence_code (mapeo exacto):\n - "presente" → "52101004"\n - "ausente" → "272519000"\n - "incierto" → "261665006"\n\n4) PROHIBIDO:\n - Elegir códigos fuera de las listas.\n - Usar fallback si hay candidatos en ENTITY.\n - Cambiar el tipo de salida.\n\nDevuelve SOLO este JSON:\n{\n "entity_code": "<SNOMED_CODE>",\n "anatomy_code": "<SNOMED_CODE>",\n "presence_code": "<SNOMED_CODE>"\n}",
"input_variables": ["entity", "location", "presence", "contexto_entity", "contexto_anatomy"],
"output_format": "json",
"expected_fields": ["entity_code", "anatomy_code", "presence_code"]
}

--- Content of ./src/prompts/filter.json ---
{
"name": "Filtrado de relevancia",
"version": "1.0",
"description": "First-pass filter to determine if RAG results contain valid matches",
"language": "es",

"template": "You are a medical terminology expert. Review the RAG search results and determine if ANY of the options are medically valid matches for the entity.\n\nEntity: {entity}\nLocation: {location}\nPresence: {presence}\n\n{contexto_entity}\n\nQUESTION: Do any of these SNOMED codes accurately represent "{entity}"?\n\nReturn JSON:\n{{\n "has_valid_match": true/false,\n "best_option_number": <1-15 or null if false>,\n "reasoning": ""\n}}",
"input_variables": ["entity", "location", "presence", "contexto_entity"],
"output_format": "json",
"expected_fields": ["has_valid_match", "best_option_number", "reasoning"]
}

--- Content of ./src/prompts/ner.json ---
{
"name": "NER Extractivo Clínico",
"version": "3.0 - Verbatim spans with offsets",
"description": "Extrae entidades como copias exactas del texto con offsets de caracteres y múltiples ocurrencias",
"language": "es",
"template": "Extrae TODAS las entidades médicas de esta nota clínica. Debes ser EXHAUSTIVO.\n\nREQUISITOS CRÍTICOS:\n- "full_span" debe ser una COPIA-PEGA EXACTA de la nota (mismos caracteres).\n- Proporciona índices de caracteres 0-based: "start" inclusivo, "end" exclusivo.\n- Si una entidad aparece VARIAS veces, crea una entrada por CADA OCURRENCIA con sus propios start/end.\n- No normalices el texto: copia exactamente lo que aparece (mayúsculas, signos, abreviaturas, etc.).\n\nPara cada entidad incluye:\n1. **core_entity**: término nuclear (p.ej., "stenosis", "weakness", "hemorrhage").\n2. **full_span**: TEXTO EXACTO tomado de la nota para esa ocurrencia.\n3. **anatomical_location**: localización específica o "No especificado".\n4. **presence**: "presente", "ausente" o "incierto".\n5. **value**: valor numérico si aplica, si no null.\n6. **start**: índice de carácter (0-based, inclusive) donde inicia "full_span".\n7. **end**: índice de carácter (0-based, exclusivo) donde termina "full_span".\n\nEXTRAER:\n- Hallazgos/patologías (hemorrhage, infarct, occlusion, stenosis, lesion, edema, mass...)\n- Anatomía (arterias, regiones cerebrales, vasos)\n- Escalas/scores (NIHSS, ASPECTS, TICI, GCS, mRS, etc.)\n- Síntomas (weakness, aphasia, numbness, paresis, paralysis)\n- Procedimientos (thrombectomy, thrombolysis, tPA, angiography, surgery)\n- Comorbilidades (AFib, hypertension, diabetes, etc.)\n- Mediciones y valores\n\nFORMATO JSON ESTRICTO:\n{\n "entities": [\n {\n "core_entity": "term",\n "full_span": "TEXTO EXACTO COPIADO",\n "anatomical_location": "location or No especificado",\n "presence": "presente|ausente|incierto",\n "value": "value or null",\n "start": 0,\n "end": 0\n }\n]\n}\n\nNota clínica:\n{informe}\n\nResponde SOLO con JSON válido:",
"input_variables": ["informe"],
"output_format": "json",
"expected_fields": ["entities"]
}

--- Content of ./src/prompts/system.json ---
{
"name": "System Prompt SNOMED-CT",
"version": "2.0",
"description": "System prompt optimizado para codificación SNOMED-CT",
"language": "es",

"content": "You are a precise SNOMED-CT coding assistant. ALWAYS select codes from the provided list. NEVER invent codes. When in doubt, choose the most specific code available. Respond ONLY with valid JSON."
}

--- Content of ./src/use_cases/01_run_ner.py ---
#!/usr/bin/env python3
"""
Use-case runner #1: NER extraction

- Input: CSV with columns [note_id, text]
- Output: JSONL, one entity per line with note_id and offsets from the input text
  """

from **future** import annotations

import argparse
import json
import sys
from pathlib import Path
import pandas as pd

Resolve src for imports when run from repo root

THIS_DIR = Path(**file**).parent.resolve()
SRC_DIR = THIS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from config import setup_openai_client, get_model_config, load_prompt
from components.ner import NERExtractor

def main():
ap = argparse.ArgumentParser()
ap.add_argument("--input", required=True, help="CSV with columns: note_id,text")
ap.add_argument("--output", required=True, help="Output JSONL path (entities per line)")
args = ap.parse_args()

```
df = pd.read_csv(args.input)
if not {"note_id", "text"}.issubset(df.columns):
    raise ValueError("Input CSV must have columns: note_id,text")

client = setup_openai_client()
ner_prompt = load_prompt("ner")
system_prompt = load_prompt("system")["content"]
model_cfg = get_model_config()
```

```
    ner = NERExtractor(client, ner_prompt, model_cfg, system_prompt=system_prompt)

    out_path = Path(args.output)
    out_path.parent.mkdir(parents=True, exist_ok=True)
    n_total = 0

    with out_path.open("w", encoding="utf-8") as f_out:
        for _, row in df.iterrows():
            note_id = row["note_id"]
            text = row["text"]
            entities = ner.extract_entities(str(text))
            for ent in entities:
                ent_out = {
                    "note_id": int(note_id),
                    "span_text": ent["span_text"],
                    "full_span": ent.get("full_span", ent["span_text"]),
                    "anatomical_location": ent.get("anatomical_location", "No
especificado"),
                    "presence": ent.get("presence", "presente"),
                    "value": ent.get("value"),
                    "start": ent.get("start"),
                    "end": ent.get("end"),
                }
                f_out.write(json.dumps(ent_out, ensure_ascii=False) + "\n")
                n_total += 1

    print(f"[01_run_ner] Wrote {n_total} entities to {out_path}")
```

if **name** == "**main**":
main()

--- Content of ./src/use_cases/02_run_rag.py ---
#!/usr/bin/env python3
"""
Use-case runner #2: RAG candidates retrieval

- Input: JSONL of entities (output of 01_run_ner.py)
- Output: JSONL of entities with RAG candidates for entity and anatomy
  """

from **future** import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Dict, List, Tuple

Resolve src for imports when run from repo root

THIS_DIR = Path(**file**).parent.resolve()
SRC_DIR = THIS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from config import get_assets_dir
from components.rag import RAGRetriever

def read_jsonl(path: Path):
with path.open("r", encoding="utf-8") as f:
for line in f:
line = line.strip()
if not line:
continue
yield json.loads(line)

def main():
ap = argparse.ArgumentParser()
ap.add_argument("--input", required=True, help="JSONL: one entity per line (from 01)")
ap.add_argument("--output", required=True, help="Output JSONL with candidates")
ap.add_argument("--k", type=int, default=30, help="Top-K candidates to retrieve")
args = ap.parse_args()

```
assets_dir = str(get_assets_dir())
rag = RAGRetriever(assets_dir)

in_path = Path(args.input)
out_path = Path(args.output)
out_path.parent.mkdir(parents=True, exist_ok=True)

n = 0
with out_path.open("w", encoding="utf-8") as f_out:
    for obj in read_jsonl(in_path):
        entity = obj.get("span_text") or obj.get("full_span") or ""
        location = obj.get("anatomical_location", "No especificado")

        ent_cands = rag.retrieve(entity, k=args.k)
        anat_cands = rag.retrieve(location, k=min(args.k, 15)) if location and
location != "No especificado" else []

        obj["entity_candidates"] = [
            {"concept_id": c, "narrative": nrr, "score": float(s)} for c, nrr,
s in ent_cands
        ]
        obj["anatomy_candidates"] = [
```

```
            {"concept_id": c, "narrative": nrr, "score": float(s)} for c, nrr,
s in anat_cands
        ]

        f_out.write(json.dumps(obj, ensure_ascii=False) + "\n")
        n += 1

print(f"[02_run_rag] Wrote {n} lines with candidates to {out_path}")
```

if **name** == "**main**":
main()

--- Content of ./src/use_cases/03_run_coding.py ---
#!/usr/bin/env python3
"""
Use-case runner #3: Coding

- Input: JSONL of entities (from 01 or 02)
- Output: JSONL of coded entities (entity_code, anatomy_code, presence_code)
  """

from **future** import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import List, Dict

Resolve src for imports when run from repo root

THIS_DIR = Path(**file**).parent.resolve()
SRC_DIR = THIS_DIR.parent
if str(SRC_DIR) not in sys.path:
sys.path.insert(0, str(SRC_DIR))

from config import setup_openai_client, load_prompt, get_model_config, get_assets_dir
from components.coding import SNOMEDCoder
from components.rag import RAGRetriever

def read_jsonl(path: Path):
with path.open("r", encoding="utf-8") as f:
for line in f:
line = line.strip()
if not line:
continue
yield json.loads(line)

def main():
ap = argparse.ArgumentParser()
ap.add_argument("--input", required=True, help="JSONL entities (from 01 or 02)")
ap.add_argument("--output", required=True, help="Output JSONL coded entities")
args = ap.parse_args()

```
client = setup_openai_client()
assets_dir = str(get_assets_dir())
rag = RAGRetriever(assets_dir)

system_prompt = load_prompt("system")["content"]
coder = SNOMEDCoder(rag, client, system_prompt=system_prompt)

in_path = Path(args.input)
out_path = Path(args.output)
out_path.parent.mkdir(parents=True, exist_ok=True)

# collect all entities as list for batch coding
entities: List[Dict] = list(read_jsonl(in_path))
coded = coder.code_entities(entities, verbose=True)

with out_path.open("w", encoding="utf-8") as f_out:
    for ent in coded:
        f_out.write(json.dumps(ent, ensure_ascii=False) + "\n")

print(f"[03_run_coding] Wrote {len(coded)} coded entities to {out_path}")
```

if **name** == "**main**":
main()

--- Content of ./src/utils/text.py ---
"""
Utilidades para procesamiento de texto y span matching
"""

import re
from typing import Tuple, Optional, List

def _make_flexible_regex(span_text: str) -> str:
"""
Construye un patrón regex flexible que ignore múltiples espacios/saltos de línea entre palabras,
escapando caracteres especiales del span.
"""
palabras = re.split(r'\s+', span_text.strip())
palabras_escapadas = [re.escape(palabra) for palabra in palabras if palabra]
if not palabras_escapadas:

return r''
return r'\s+'.join(palabras_escapadas)

def find_span_in_text(span_text: str, text: str, start_from: int = 0) -> Optional[Tuple[int, int]]:
"""
Encuentra la primera posición de un span en el texto usando regex flexible (case-insensitive).

```
Args:
    span_text: Texto del span a buscar
    text: Texto completo donde buscar
    start_from: Índice desde donde comenzar la búsqueda

Returns:
    Tupla (start, end) si se encuentra, None si no
"""
pattern = _make_flexible_regex(span_text)
if not pattern:
    return None

match = re.search(pattern, text[start_from:], re.IGNORECASE)
if match:
    start = match.start() + start_from
    end = match.end() + start_from
    return (start, end)
return None
```

def find_all_spans_in_text(span_text: str, text: str) -> List[Tuple[int, int]]:
"""
Encuentra TODAS las ocurrencias de un span en el texto con regex flexible (case-insensitive).

```
Args:
    span_text: Texto del span a buscar
    text: Texto completo

Returns:
    Lista de tuplas (start, end) para cada coincidencia encontrada
"""
pattern = _make_flexible_regex(span_text)
if not pattern:
    return []

matches = []
for m in re.finditer(pattern, text, re.IGNORECASE):
    matches.append((m.start(), m.end()))
return matches
```

def clean_json_response(response: str) -> str:
"""
Limpia una respuesta JSON de markdown y trailing commas

```
Args:
    response: Respuesta cruda del LLM

Returns:
    JSON limpio como string
"""
response_clean = response.strip()

# Remover markdown
if '```json' in response_clean:
    json_start = response_clean.find('```json') + 7
    json_end = response_clean.find('```', json_start)
    response_clean = response_clean[json_start:json_end].strip()
elif '```' in response_clean:
    json_start = response_clean.find('```') + 3
    json_end = response_clean.find('```', json_start)
    response_clean = response_clean[json_start:json_end].strip()

# Limpiar trailing commas
response_clean = re.sub(r',(\s*[}\]])', r'\1', response_clean)

return response_clean
```

==========

NUEVAS UTILIDADES (EXACT MATCH)

==========

def find_exact_span(span_text: str, text: str) -> Optional[Tuple[int, int]]:
"""
Búsqueda EXACTA (case-sensitive) de un span en todo el texto.
Devuelve solo la PRIMERA ocurrencia exacta.
"""
if not span_text:
return None
idx = text.find(span_text)
if idx == -1:

return None
return (idx, idx + len(span_text))

def find_first_case_insensitive(span_text: str, text: str) -> Optional[Tuple[int, int]]:
"""
Búsqueda global de la PRIMERA ocurrencia en modo case-insensitive.
Devuelve offsets en el texto original.
"""
if not span_text:
return None
idx = text.lower().find(span_text.lower())
if idx == -1:
return None
return (idx, idx + len(span_text))

def find_exact_span_near(span_text: str, text: str, approx_start: int, window: int = 50) -> Optional[Tuple[int, int]]:
"""
Busca un span EXACTO alrededor de un offset aproximado (ventana local).
Primero intenta case-sensitive, luego case-insensitive. Devuelve UNA coincidencia.
"""
if not span_text:
return None
start = max(0, approx_start - window)
end = min(len(text), approx_start + window + len(span_text))

```
# Case-sensitive primero
idx = text.find(span_text, start, end)
if idx != -1:
    return (idx, idx + len(span_text))

# Case-insensitive como último recurso local
lower_text = text.lower()
lower_span = span_text.lower()
idx2 = lower_text.find(lower_span, start, end)
if idx2 != -1:
    return (idx2, idx2 + len(span_text))

return None
```

==========

NUEVO: Recorte de bordes

==========

_PUNCT_TO_TRIM = set(list(' \t\r\n.,;:!?"'()[]{}／／\'))

def tighten_span_boundaries(text: str, start: int, end: int) -> Tuple[int, int]:
"""
Recorta espacios y puntuación en los bordes de un span sin pasar del centro.
- Evita incluir whitespace/puntuación que los anotadores suelen excluir.
- Garantiza que end > start si había al menos 1 carácter no-espacio.
"""
if not isinstance(start, int) or not isinstance(end, int):
return start, end
s, e = max(0, start), min(len(text), end)

```
# trim left
while s < e and (text[s].isspace() or text[s] in _PUNCT_TO_TRIM):
    s += 1
# trim right
while e > s and (text[e - 1].isspace() or text[e - 1] in _PUNCT_TO_TRIM):
    e -= 1

return s, e
```

--- Content of ./src/utils/**init**.py ---

Marks utils as a package

--- Content of ./src/**init**.py ---

This file marks 'src' as a package